LKML5Ws: The What, When, Who, Where, and Why in the Linux Kernel Mailing Lists - Preprint

Rafael Passos, Arthur Pilone, David Tadokoro, Laura Arakaki, Paulo Meirelles
{rcpassos,arthurpilone,davidbtadokoro,laura.arakaki,paulormm}@ime.usp.br
Institute of Mathematics, Statistics, and Computer Science, University of São Paulo
São Paulo, Brazil

***Abstract*—The Linux kernel uses mailing lists as the primary medium for all development, bug reporting, and pivotal discussions regarding the project's future, a practice also observed in other large and long-lived Free Software projects. However, as a consequence of the decentralized development model used for the Linux kernel, these emails are spread across hundreds of mailing lists, each with different communities and maintenance models. We present the LKML5Ws dataset and our MailingListsHeritage tool, developed to create and maintain mailing list archives. With almost 30 million emails from 350 different mailing lists, our massive relational dataset provides a comprehensive overview of the last 30 years of Linux kernel development. Beyond shedding light on the awe-inspiring number of patches, discussions, and contributors involved in the project, our dataset provides a foundation for those interested in studying the intricate, knowledge-rich nature of Linux kernel development.**

***Index Terms*—Linux kernel, mailing lists, Free Software development, Free Software maintainers**

I. Introduction

For decades, software engineering researchers have turned to the Linux kernel to study whether their tools, techniques, and theories apply when scaled up to one of the most extensive and emblematic Free Software projects [1], [2], [3]. With tens of millions of LoC written through the combined effort of generations of contributors for over 30 years, the size and longevity of the Linux kernel also contribute to some of its defining particularities: the decentralized development model involving dozens of subprojects (called *subsystems*) organized in an implicit hierarchical structure [4], and the use of mailing lists as the primary means of communication.

Ranging from enduring high-traffic lists to quieter, now-defunct ones, these public archives encompass tens of millions of messages. Much of this content includes information absent from *version control systems* (VCSs): praise that secured contributions and critiques that blocked others. As such, these lists are an invaluable source for understanding the reasoning behind the contributions and decisions that shape the project.

While contemporary research on mining software repositories thrives on the prompt parsing nature of VCS data, typically Git, mining Linux mailing lists requires intermediate knowledge of its development model and careful parsing of the textual formats of emails. Important information, such as the `Reviewed-by` tag that acknowledges community members who contributed to a review, is poorly structured and could be overlooked by naive mining techniques.

We present the LKML5Ws dataset, a comprehensive and extensive collection of contributions extracted from emails sent to the Linux kernel mailing lists. For every contribution in our dataset, we share: (*i)* the code modification *(what)*; (*ii)* the date the contribution was sent *(when)*; (*iii)* the roles of the auxiliary contributors tagged on the contribution *(who)*; (*iv)* the list the contribution was sent to *(where)*; and (*v)* the authors' explanation of the given contribution as read from the email discussion that accompanies it *(why)*.

In addition to the dataset, we also share the tool we developed to create it. Including almost 30 million emails from 350 different mailing lists, our dataset should help future work exploring the lengthy, intensely collaborative, and nuanced process of reviewing contributions in the Linux kernel.

II. Background

While the mailing lists may have originally been used simply because they were the most accessible means of online communication at the time the project started, they remain the official channels for communication and patch submissions in the Linux kernel development. Current key maintainers argue that mailing lists remain the most suitable solution for the vast and decentralized project [5].

Such decentralization manifests as the division of responsibility among multiple *subsystems*, with most having a dedicated Git repository (called *kernel tree*) and a set of maintainers responsible for managing their slice of the codebase and reviewing contributions related to it. Each contribution, also known as a *patch*, is sent via email to the subsystem's mailing list [6]. Mailing lists are generally open, allowing anyone interested in contributing to subscribe to the development lists and participate in the review process. Patches are very similar to Git commits, as they encode the proposed change (in a format resembling a `git diff`), and include a descriptive title and message, along with the signatures of the community members involved in the contribution.

Given the open nature of the mailing lists, it is common for non-maintainer community members to contribute to the patch review process. They can support a patch that awaits a maintainer's approval by testing or reviewing it themselves. In this case, one can register the approval of these auxiliary contributors by tagging them in the patch. Tags such as `Tested-by`, `Reviewed-by`, and `Acked-by` are commonly used to give credit to members of the community [7].

Preprint: paper accepted to IEEE ICSME 2026, Tool Demonstration and Data Showcase Track. DOI: pending

The official way to access these mailing lists, besides email subscription, is through the *Kernel Lore* website[1]. This official archive currently contains emails from 350 lists related to the Linux kernel and other correlated projects, such as the QEMU virtualization tool and Git itself.

III. Data Collection and Processing

The *Kernel Lore* is the official archive that preserves all emails exchanged on the mailing lists covering the Linux kernel development process. This archive is an instance of `Public-Inbox`. It archives emails in Git repositories, and optionally exposes them with NNTP, IMAP, POP3, Atom feeds, or HTML.

The official mirroring instructions provided by the `Public-Inbox` software[2] recommend two options: the NNTP Protocol, and cloning the underlying `Public-Inbox` Git repositories directly. There is also the `grokmirror` tool, developed by a Linux kernel maintainer, to optimize the initial clone and recurrent sync operations. We host a mirror of all the *Lore* data, and use `grokmirror` to keep it up to date. Our mirroring setup is available online.[3]

Our tool, `MailingListsHeritage`, has four main components: *(i) Archiver*: collects raw emails from the configured sources; *(ii) Parser*: extracts information from email headers and body; *(iii) Anonymizer*: applies hash-based pseudonymization; *(iv) Analysis*: tests and validates the resulting files.

Archiver. The first component can read data from both the NNTP interface and `Public-Inbox` Git. It was built to be extensible, and a new source sub-module must only implement a simple interface (a Rust Trait). For our dataset, we use the `Public-Inbox` method, as it is more efficient. The archiver also produces a data-lineage artifact: for each row in the dataset, it stores details about the configurations and the software version used. The output is a folder per list, with raw email files or parquet files to reduce IO overhead.

Parser. Since we are interested in organizing the data in a queryable format, we parse the headers and the body of each email, extracting and normalizing each data point into a field. From the email headers, we extract the sender and the recipients (`from`, `to` and `cc` fields), `subject`, the date the email was sent (`date` and `client_date`), the message id, also used in replies and threads (`message_id`, `in_reply_to` and `references`), and the target mailing list. From the subject, we extract "tags" used as convention (*i.e.,* "RFC", "PATCH") if it includes a patch version, if the patch is part of a patch-set, and if the email is a response. These columns have a name with a "_subject_tag" suffix.

We save each email body in its entirety as the `raw_body` column and the SHA1 hash as `body_sha1`. When an email sent is a patch, *i.e.,* a code contribution, we store the diff with the modification in the `code` column. In the Kernel, there is also the standard practice of using a "Fixes" line when fixing a bug with a known cause, and it indicates the commit that introduced the error. This is collected into the `fixes_body_tags` column. We also identify and store any signatures and tags (*e.g.,* `Signed-off-by`, `Reviewed-by`, `Acked-by`) present in the email body, saving them in the `trailers` column.

The parser also accepts a git mailmap file as an optional input. This file maps identities and emails that change over time. In the Linux Kernel, this is used by maintainers and is available in the git tree. This file is used to mirror values from the "from" and "trailers" with the "_canonical_addr" suffix, when the emails are found in the map.

Parsing the dates from the emails in our dataset posed its own challenges, as the author's email client provides the date field. Flint and Dyer [8] also raised concerns about bad data input in timestamp fields. The authors argue that a simple invalid date cutoff would suffice for most cases. In our tool, we store the raw user-declared date in the `client_date` field. For the `date` field, we also look for other headers (*e.g.,* receive date from servers and relays), and to determine the final value, we use the median absolute deviation to exclude outliers and select the earliest valid date. In addition to the parsed data, a lineage file is created that contains metadata about the archiving and parsing processes.

Anonymizer. To preserve the privacy of the Linux development community, we pseudonymize all identities from our parsed messages. We compute the SHA-1 hashes for every name or email in the `from`, `to`, `cc`, `trailers`, and `raw_body` fields. In addition, we create a validation dataset that maps the found identities to their hashes. We chose this pseudonymization strategy to align with the one used by the Software Heritage project in its public datasets.[4]

Analysis. We have a dedicated component for data validation. We look for data duplication (inherited from the mailing lists), missing dates, and missing code (when code is expected). In addition, we share a few exploratory analyses for comparing lists, checking the author distribution in the lists, and running arbitrary SQL queries on the dataset. A detailed documentation of the complete tool is available in its repository on Software Heritage [9].

IV. Leveraging the LKML5Ws Dataset

Our massive dataset encompasses almost 30 million emails. Half of our 350 mailing lists have over 14,000 emails, with the 25% most active lists containing over 55,000 emails each. This significant volume of data can help us explore rich comparisons between kernel subsystems or conduct in-depth investigations into some fundamental characteristics of how the Linux kernel is developed and maintained.

Due to the common practice of sending the same emails to more than one mailing list, our dataset inherited data duplication from the source data. Out of its 30 million emails, 20 million are unique; 5 million emails have at least one replica, with 36% of them having more than one.

[1] See https://lore.kernel.org/

[2] See https://lore.kernel.org/lkml/_/text/mirror/

[3] See https://gitlab.com/ccsl-usp/codev/Public-Inbox-Stack

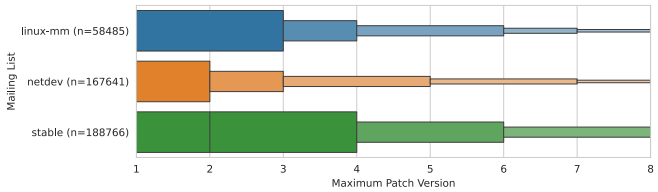
[4] See https://docs.softwareheritage.org/user/using_data/index.html

Preprint: paper accepted to IEEE ICSME 2026, Tool Demonstration and Data Showcase Track. DOI: pending

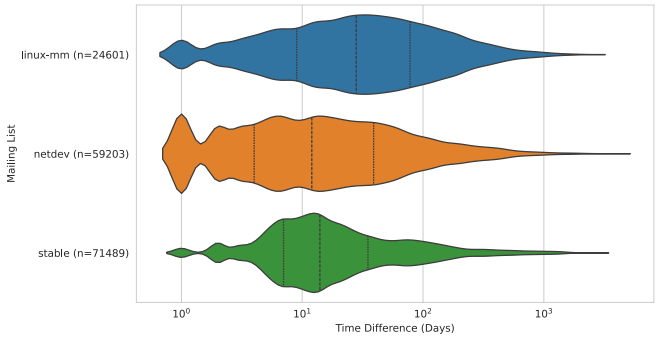
A. Exploring the Linux Maintainership Model

Over the last decade, the Linux kernel community has raised concerns about potential scalability issues with the current maintainership model adopted for the project's development [10], [11]. Recent multivocal literature reviews suggest that these scalability issues may be more than just the personal experience of a few contributors and part of a deeper problem within the Linux development process [12], [13], [14].

In Figure 1, we observe the last ten years (2016 to 2026) of three distinct lists "linux-mm", the Memory Management list, "netdev" for networking, and "stable" for maintenance of the stable Linux releases. In subfigure (a), we present the distribution of maximum patch versions per list. In (b), we observe the time difference between the first and last versions of patches that have at least a second version.



(a) Distribution of the maximum number of patch versions.



(b) Latency between the first and last patch versions.

Fig. 1. Analysis of patch versions and latencies per list.

Patches that receive a revision require rework, and new reviews from the maintainers. There are also ignored patches, usually re-posted afterward, as noted by Rigby *et al.* [15]. In (a), we see the extra effort involved in maintaining the stable Linux releases: 90% of patches in the stable list have up to 7 versions, while in netdev, linux-mm, and also across all lists, patches have up to 4 versions. In contrast, the time between these revisions is shorter (b), as expected, given that they are patches backported from the mainline. **This data cannot be extracted by mining VCS data**, because only the accepted version of each patch is committed to Git.

B. Prospective Research Questions

Sizewise, we understand that the sheer volume of emails in our dataset provides a solid basis for studies on communication between developers, or between developers and maintainers/reviewers. Future work may investigate the prevalence and impact of uncivil comments on the kernel mailing lists [16], [17] (*Is there a correspondence between incivility and the scarcity of maintainers in a given list? Is incivility falling or on the rise?*), or explore which issues are most commonly pointed out by reviewers [18], [19] (*Is code duplication acceptable? Is complexity as significant of an issue as legibility?*).

Section IV-A serves as an example of the type of analysis that particularly interests us. We understand that one of the most valuable contributions of LKML5Ws stems from the three-fold division of each email body into a central section, a trailer section, and a code section.

Following the motivation that also guided our previous work (DUKS [7]), future studies can compare metrics across different kernel trees to analyze the impact of the maintainership models adopted in different parts of the kernel. Specifically, *which subsystems have the fewest reviewing contributors per patch received? Which subsystems have the highest ratio of lines of code changed to active maintainers? Does the average length of the patch body change between trees?*

V. Future Improvements

We expect to reduce the impact of duplicate emails in our dataset with a future extension. Specifically, we will merge the information from the mailing lists in our dataset with the version history data collected from the Software Heritage archive [20]. In addition to including VCS information for the accepted patches, this approach will leverage the hash-based indexing model provided by Software Heritage. The archive's source code files, for example, are indexed by their hash digests. This indexing scheme prevents the storage of duplicate content and, when applied to the entire body of emails, prevents duplications from being stored in the dataset.

We aim to fix more inaccurate dates. As an additional technique in the parser, we want to compare the `date` value with dates in quoted text and with dates from other emails linked by `message-id` headers.

Future extensions of our dataset can also leverage the high flexibility of our email collection approach to include emails from other large Free Software projects. While our tool was built with the LKML in mind, it can collect emails from other sources that use `Public-Inbox` or have an NNTP endpoint. As a test, we read the Debian community mailing lists using NNTP and collected 31K emails from 65 different lists.

VI. Related Work

Given the long-standing respect that software engineering researchers have for the Linux kernel, numerous prior works have explored it. From characterizing Free Software development [21], [22] to serving as an ideal case study for software analysis tools [2], [3], the kernel has amply served the software engineering community.

Additionally, the use of mailing lists as the primary means of communication in Free Software projects has also been the target of previous work [23], [24]. Passos and Czarnecki [25], as well as German *et al.* [26], created large datasets capturing the evolution of the Linux kernel codebase. While Passos and Czarnecki bring a unique feature-oriented perspective, German

Preprint: paper accepted to IEEE ICSME 2026, Tool Demonstration and Data Showcase Track. DOI: pending

et al. focus on aggregating Git data from multiple Git trees into a single unified Git repository, which they call the Git *super-repository* of Linux. However, neither Passos and Czarnecki nor German *et al.* explore the Linux mailing lists.

The work of Xu and Zhou [27] is particularly relevant to ours. These authors created a dataset of over 660,000 emailed patches, stored on the Linux Patchwork website,[5] alongside their corresponding Git commits. However, restricting their scope to the Patchwork archive limited this previous dataset to fewer than 120 mailing lists and only 9 years of emails. In contrast, our LKML5Ws dataset collects data across 350 mailing lists, spanning 30 years of development. Furthermore, while their work omits auxiliary signatures and restricts its scope strictly to patch threads, our dataset parses these signatures and incorporates broader discussion threads.

VII. Ethical Considerations

Given the scale of our dataset and the contributors directly or indirectly included within it, addressing ethical considerations is of utmost importance [28], [29]. Following Gold and Krinke [28], we structure our ethical considerations according to the four principles from the Menlo Report [6]:

Identification of stakeholders and informed consent: Since our dataset includes decades of development spread across multiple mailing lists, any developer who has ever contributed to one of the collected lists could be included. The significant temporal aspect makes obtaining individual consent impossible, as some contributors have passed away. However, as noted in the official Linux kernel documentation,[7] each author signing a contribution to any mailing list agrees to the Developer Certificate of Origin.[8] By doing so, they acknowledge that all information sent with the contribution is publicly available and redistributable in accordance with the license adopted for the kernel.

Balancing risks and benefits: Since all data included in our dataset is publicly available, we understand that the most substantial risk associated with it does not come from the data itself, but from the large-scale aggregation of sensitive information. As such, and as described in Section III, we pseudonymize names and email addresses within our dataset.

Fairness and equity: By including as many mailing lists as possible, we minimize the risk of biasing our data toward specific subcommunities of the kernel. We note that our dataset naturally includes more information from the most active contributors to the lists hosted in the *Kernel Lore* archive.

Compliance, transparency, and accountability: We share the dataset, along with the tool we used to create it, under the GNU General Public License (GPLv3); the Linux kernel itself is licensed under GPLv2.

[5]See https://patchwork.kernel.org/

[6]https://www.dhs.gov/sites/default/files/publications/CSD-MenloPrinciplesCORE-20120803_1.pdf

[7]See https://www.kernel.org/doc/html/latest/process/submitting-patches.html#sign-your-work-the-developer-s-certificate-of-origin

[8]See https://developercertificate.org/

VIII. Conclusion

We built the LKML5Ws dataset, collected using our `MailingListsHeritage` tool. Our extensive collection of almost 30 million emails encompasses 350 mailing lists hosted in the *Kernel Lore* archive, offering a comprehensive view of how the Linux kernel is developed. By highlighting *what* each contribution consists of, *when* it was first envisioned, *who* helped adjust and approve it, *where* it was sent to, and *why* it became part of the kernel code (or failed to do so), our dataset supports future research into the intricate details that define the most emblematic Free Software project to date.

Dataset and Tool

The dataset is archived in Zenodo, under the GPLv3 License *https://doi.org/10.5281/zenodo.17567225*. The source code for `MailingListsHeritage`, along with its usage instructions, is archived on Software Heritage [9]. The complete reproduction package is archived in Zenodo at *https://doi.org/10.5281/zenodo.20434788*.

Acknowledgments

This study was financed, in part, by CAPES (Finance Code 001), and the São Paulo Research Foundation – FAPESP (Procs. 2025/26679-7, 2025/05395-0) and the São Paulo State Data Analysis System Foundation – SEADE (FAPESP Proc. 2023/18026-8), Brazil.

References

[1] I. T. Bowman, R. C. Holt, and N. V. Brewster, "Linux as a case study: Its extracted software architecture," in *Proceedings of the International Conference on Software Engineering (ICSE)*, ACM, 1999, pp. 555–563.

[2] R. Suvorov, M. Nagappan, A. E. Hassan, Y. Zou, and B. Adams, "An empirical study of build system migrations in practice: Case studies on kde and the linux kernel," in *Proceedings of the International Conference on Software Maintenance (ICSM)*, IEEE, 2012, pp. 160–169.

[3] G. Occhipinti, C. Nagy, R. Minelli, and M. Lanza, "Syn: Ultra-scale software evolution comprehension," in *Proceedings of the International Conference on Program Comprehension (ICPC)*, IEEE, 2023, pp. 69–73.

[4] J. Corbet. "Patch flow into the mainline for 4.14." [Online; Last accessed on Nov. 10th, 2025], Linux Weekly News. [Online]. Available: https://lwn.net/Articles/737093/

[5] J. Corbet. "Why kernel development still uses email." [Online; Last accessed on Oct. 31st, 2025], Linux Weekly News. [Online]. Available: https://lwn.net/Articles/702177/

[6] *How the development process works: Mailing lists*, [Online; Last accessed on Nov. 10th, 2025], The kernel development community. [Online]. Available: https://www.kernel.org/doc/html/v6.17/process/2.Process.html#mailing-lists

Preprint: paper accepted to IEEE ICSME 2026, Tool Demonstration and Data Showcase Track. DOI: pending

[7] R. Passos, A. Pilone, D. Tadokoro, and P. Meirelles, "Streamlining analyses on the linux kernel with duks," in *Proceedings of the Working Conference on Software Visualization (VISSOFT)*, IEEE, 2025, pp. 125–128.

[8] S. W. Flint, J. Chauhan, and R. Dyer, "Escaping the Time Pit: Pitfalls and Guidelines for Using Time-Based Git Data," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, IEEE, May 2021, pp. 85–96. Accessed: Jan. 4, 2026.

[9] [SW Rel.] R. Passos, A. Pilone, L. Arakaki, D. Tadokoro, and P. Meirelles, *MailingListsHeritage* version 1, Oct. 11, 2025, University of São Paulo, LIC: GPL-3.0, VCS: https://gitlab.com/ccsl-usp/codev/MailingListsHeritage.git, SWHID: ⟨swh:1:snp:3103dd1b9a83152b678b5a45c0e7c1c685d00017;origin=https://gitlab.com/ccsl-usp/codev/MailingListsHeritage⟩.

[10] D. Vetter, *Maintainers don't scale*, [Online; Last accessed on Nov. 5th, 2025], 2017. [Online]. Available: blog.ffwll.ch/2017/01/maintainers-dont-scale.html

[11] S. Dean, *The maintainer's paradox: Balancing project and community*, [Online; Last accessed on Nov. 5th, 2025], Linux Foundation, 2020.

[12] D. Tadokoro, R. Siqueira, and P. Meirelles, "Can the linux kernel sustain 30 more years of growth? toward mitigating bottlenecks in its development model," in *Proceedings of the Brazilian Symposium on Software Engineering (SBES)*, SBC, 2025, pp. 845–851.

[13] E. Pinheiro and P. Meirelles, "Understanding group maintainership model in the linux kernel development," in *Proceedings of the Workshop on Software Visualization, Maintenance and Evolution (VEM)*, SBC, 2024, pp. 113–124.

[14] M. S. R. Wen, "What happens when the bazaar grows: A comprehensive study on the contemporary linux kernel development model," Ph.D. dissertation, University of São Paulo, 2021.

[15] P. C. Rigby, D. M. German, L. Cowen, and M.-A. Storey, "Peer Review on Open-Source Software Projects: Parameters, Statistical Models, and Theory," Sep. 5, 2014.

[16] R. Ehsani, M. M. Imran, R. Zita, K. Damevski, and P. Chatterjee, "Incivility in open source projects: A comprehensive annotated dataset of locked github issue threads," in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, ACM, 2024, pp. 515–519.

[17] D. Gachechiladze, F. Lanubile, N. Novielli, and A. Serebrenik, "Anger and its direction in collaborative software development," in *Proceedings of the International Conference on Software Engineering: New Ideas and Emerging Results Track (ICSE-NIER)*, IEEE, 2017, pp. 11–14.

[18] P. W. Gonçalves, P. Rani, M.-A. Storey, D. Spinellis, and A. Bacchelli, "Code review comprehension: Reviewing strategies seen through code comprehension theories," in *Proceedings of the International Conference on Program Comprehension (ICPC)*, 2025, pp. 589–601.

[19] M. S. Rahman, Z. Codabux, and C. K. Roy, "Investigating the understandability of review comments on code change requests," in *Proceed of the International Conference on Mining Software Repositories (MSR)*, IEEE, 2025, pp. 539–551.

[20] A. Pietri, D. Spinellis, and S. Zacchiroli, "The Software Heritage Graph Dataset: Public Software Development Under One Roof," en, in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, IEEE, 2019, pp. 138–142. Accessed: Apr. 4, 2025.

[21] G. Avelino, L. Passos, A. Hora, and M. T. Valente, "Assessing code authorship: The case of the linux kernel," en, in *Open Source Systems: Towards Robust Practices*, vol. 496, Springer, 2017, pp. 151–163.

[22] E. Dias, P. Meirelles, F. Castor, I. Steinmacher, I. Wiese, and G. Pinto, "What makes a great maintainer of open source projects?" In *Proceedings of the International Conference on Software Engineering (ICSE)*, IEEE, 2021, pp. 982–994.

[23] A. Guzzi, A. Bacchelli, M. Lanza, M. Pinzger, and A. v. Deursen, "Communication in open source software development mailing lists," in *Proceedings of the Working Conference on Mining Software Repositories (MSR)*, IEEE, 2013, pp. 277–286.

[24] D. Schneider, S. Spurlock, and M. Squire, "Differentiating Communication Styles of Leaders on the Linux Kernel Mailing List," en, in *Proceedings of the 12th International Symposium on Open Collaboration*, ACM, 2016, pp. 1–10. Accessed: Apr. 20, 2025.

[25] L. Passos and K. Czarnecki, "A dataset of feature additions and feature removals from the linux kernel," in *Proceedings of the Working Conference on Mining Software Repositories (MSR)*, ACM, 2014, pp. 376–379.

[26] D. M. German, B. Adams, and A. E. Hassan, "A dataset of the activity of the git super-repository of linux in 2012," in *Proceedings of the Working Conference on Mining Software Repositories (MSR)*, IEEE, 2015, pp. 470–473.

[27] Y. Xu and M. Zhou, "A multi-level dataset of linux kernel patchwork," in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, IEEE, 2018, pp. 54–57.

[28] N. E. Gold and J. Krinke, "Ethical mining: A case study on msr mining challenges," en, in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, ACM, 2020, pp. 265–276.

[29] G. Robles, L. Arjona Reina, A. Serebrenik, B. Vasilescu, and J. M. González-Barahona, "Floss 2013: A survey dataset about free software contributors: Challenges for curating, sharing, and combining," en, in *Proceedings of the Working Conference on Mining Software Repositories (MSR)*, ACM, 2014, pp. 396–399.

Preprint: paper accepted to IEEE ICSME 2026, Tool Demonstration and Data Showcase Track. DOI: pending